Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №3
по дисциплине «Логика и основы алгоритмизации в инженерных задачах»
на тему «Динамические списки»

Выполнили ст. гр. 25ВВВ1:
Мингалеева А. Р.
Андреев М. А.

Приняли:
к.т.н., доцент Юрова О.В.
ст., преподаватель Деев М.В.

Пенза 2026

1. Цель работы

Изучить принцип работы динамических списков и особенности их реализации с использованием указателей; освоить создание и организацию элементов односвязного списка в динамической памяти, а также основные операции над списком: добавление, удаление и поиск элементов.

2. Лабораторное задание

1. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).
2. На основе приведенного кода реализовать структуру данных *Очередь*.
3. На основе приведенного кода реализовать структуру данных *Стек*.

3. Ход работы

3.1. Описание используемых алгоритмов

- Алгоритм создания элемента динамического списка:
  Для создания нового элемента используется структура Node, содержащая информационное поле, приоритет (для приоритетной очереди) и указатель на следующий элемент списка. Память под новый элемент динамически выделяется с помощью оператора new. После заполнения информационных полей указатель next устанавливается в значение nullptr. Созданный элемент возвращается в вызывающую функцию.

- Алгоритм работы с динамической памятью:
  Элементы списков создаются во время выполнения программы с использованием оператора new. Каждый новый элемент размещается в динамической памяти и связывается с другими элементами посредством указателя next. При удалении элементов используется оператор delete. Для предотвращения утечек памяти предусмотрены отдельные функции очистки списков, которые последовательно удаляют все элементы.

- Алгоритм добавления элемента в приоритетную очередь:
  При добавлении нового элемента сначала проверяется наличие элементов в очереди. Если очередь пуста, новый элемент становится первым. Если приоритет нового элемента выше приоритета первого элемента, новый элемент помещается в начало списка. В противном случае выполняется последовательный проход по списку до нахождения подходящего места. Новый элемент вставляется таким образом, чтобы элементы списка располагались в порядке убывания приоритета. Элемент с большим приоритетом располагается перед элементом с меньшим приоритетом.

- Алгоритм удаления элемента из приоритетной очереди:
  Удаление выполняется с начала списка. Так как элементы предварительно упорядочены по приоритету, первым удаляется элемент с наивысшим приоритетом. Указатель head перемещается на следующий элемент списка, после чего память удалённого элемента освобождается оператором delete.

- Алгоритм работы обычной очереди:
  Для реализации обычной очереди используются указатели queueHead и queueLast, указывающие соответственно на первый и последний элементы. При добавлении новый элемент помещается в конец очереди, а указатель queueLast перемещается на него. При удалении удаляется первый элемент очереди, после чего указатель queueHead перемещается на следующий элемент. Таким образом, реализуется принцип FIFO — «первым вошёл, первым вышел».

- Алгоритм работы стека:
  Стек реализован с использованием односвязного списка и указателя stackHead. При добавлении нового элемента он помещается в начало списка и становится вершиной стека. При удалении удаляется элемент, на который указывает stackHead, после чего указатель перемещается на следующий элемент. Таким образом, реализуется принцип LIFO — «последним вошёл, первым вышел».

- Алгоритм просмотра списка:
  Для вывода содержимого списка используется указатель, который устанавливается на первый элемент. Затем выполняется последовательный переход от одного элемента к следующему с помощью указателя next. Обход продолжается до тех пор, пока текущий указатель не станет равен nullptr. Для каждого элемента выводится содержащаяся в нём информация.

- Алгоритм поиска элемента:
  Поиск осуществляется последовательным перебором элементов односвязного списка. Информационное поле текущего элемента сравнивается с введённым пользователем значением с помощью функции strcmp(). При совпадении найденный элемент возвращается функцией. Если конец списка достигнут и совпадение не найдено, возвращается значение nullptr и выводится сообщение об отсутствии элемента.

- Алгоритм удаления элемента:
  При удалении элемента указатели списка изменяются таким образом, чтобы исключить удаляемый элемент из цепочки. После изменения связей память, занимаемая удалённым элементом, освобождается с помощью оператора delete. Для каждого типа структуры предусмотрена отдельная функция удаления элементов.

4. Результат работы программы

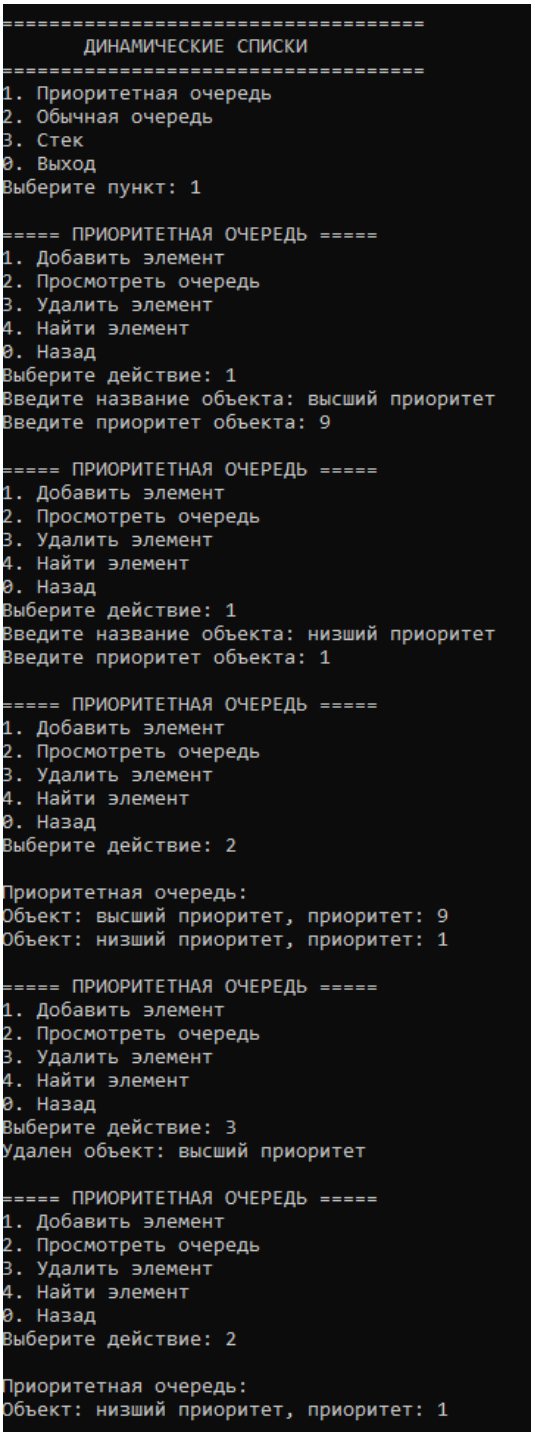


Рисунок 1 – Приоритетная очередь

```
===== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ =====
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент
4. Найти элемент
0. Назад
Выберите действие: 4
Введите название объекта: а

Элемент не найден.

===== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ =====
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент
4. Найти элемент
0. Назад
Выберите действие: 4
Введите название объекта: низший приоритет

Элемент найден.
Название: низший приоритет
Приоритет: 1
```

Рисунок 2 – Приоритетная очередь

```
===== ОБЫЧНАЯ ОЧЕРЕДЬ =====
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент
4. Найти элемент
0. Назад
Выберите действие: 1
Введите название объекта: 5

===== ОБЫЧНАЯ ОЧЕРЕДЬ =====
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент
4. Найти элемент
0. Назад
Выберите действие: 2

Очередь:
Объект: 7
Объект: 3
Объект: 7
Объект: 5

===== ОБЫЧНАЯ ОЧЕРЕДЬ =====
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент
4. Найти элемент
0. Назад
Выберите действие: 3
Удален объект: 7

===== ОБЫЧНАЯ ОЧЕРЕДЬ =====
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент
4. Найти элемент
0. Назад
Выберите действие: 4
Введите название объекта: 5
Элемент найден: 5
```

Рисунок 3 – Обычная очередь

```
===== ОБЫЧНАЯ ОЧЕРЕДЬ =====
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент
4. Найти элемент
0. Назад
Выберите действие: 4
Введите название объекта: 1
Элемент не найден.
```

Рисунок 4 – Обычная очередь

```
===== СТЕК =====
1. Добавить элемент (push)
2. Просмотреть стек
3. Удалить элемент (pop)
4. Найти элемент
0. Назад
Выберите действие: 1
Введите название объекта: mogg

===== СТЕК =====
1. Добавить элемент (push)
2. Просмотреть стек
3. Удалить элемент (pop)
4. Найти элемент
0. Назад
Выберите действие: 2

Стек:
Объект: mogg
Объект: сигма
Объект: 300
Объект: двести

===== СТЕК =====
1. Добавить элемент (push)
2. Просмотреть стек
3. Удалить элемент (pop)
4. Найти элемент
0. Назад
Выберите действие: 3
Удален объект: mogg

===== СТЕК =====
1. Добавить элемент (push)
2. Просмотреть стек
3. Удалить элемент (pop)
4. Найти элемент
0. Назад
Выберите действие: 4
Введите название объекта: 200
Элемент не найден.

===== СТЕК =====
1. Добавить элемент (push)
2. Просмотреть стек
3. Удалить элемент (pop)
4. Найти элемент
0. Назад
Выберите действие: 4
Введите название объекта: двести
Элемент найден: двести
```

Рисунок 5 – Стек

5. Вывод

В ходе лабораторной работы были изучены динамические односвязные списки и работа с указателями в C++. Были реализованы приоритетная очередь, обычная очередь и стек с основными операциями добавления, удаления, поиска и просмотра элементов.